

A RAG-Based Conversational Agent for Personalized E-Commerce Recommendations

Joanne Faith Cabarde

University of Mindanao, Matina, Davao City, Philippines

j.cabarde.548077@umindanao.edu.ph

Abstract - This study introduces Veridian, an AI-powered fashion assistant designed to enhance personalized product discovery in e-commerce. Leveraging a hybrid Retrieval-Augmented Generation (RAG) architecture, the system combines local semantic vector embeddings with Meta's llama-3.1-8b-instant model via the Groq API to deliver accurate, context-aware recommendations grounded in a curated clothing dataset. Using Pinecone for high-speed semantic retrieval and Groq's Language Processing Units (LPUs) for generative inference, Veridian eliminates the latency bottlenecks of local hardware. Furthermore, it mitigates model hallucinations by employing a strict "Escape Hatch" prompt engineering strategy, constraining responses to verified catalog data. Deployed on the Streamlit Community Cloud, the system enables real-time, personalized interactions, demonstrating the potential of cloud-accelerated RAG frameworks for scalable, reliable digital retail solutions.

Keywords - *Retrieval-Augmented Generation, Large Language Models, Conversational AI, E-commerce, Chatbot, Vector Database, Semantic Search*

1. Introduction

The global e-commerce landscape has undergone a radical transformation, moving from simple digital catalogs to complex, AI-driven ecosystems. Despite this

evolution, a persistent gap remains between user intent and search results. Conventional search engines rely heavily on keyword matching, which often fails to decipher the "latent intent" behind natural language queries. For instance, a user searching for "breathable office wear for a humid climate" might be poorly served by a system that only looks for the individual keywords "office" and "wear." Large Language Models (LLMs) have emerged as a potent solution to this problem due to their advanced natural language understanding (NLU) capabilities [2].

However, deploying standard LLMs in a retail environment presents two significant risks: hallucination and knowledge obsolescence. An LLM might confidently recommend a high-end designer jacket that a specific retailer does not actually stock, leading to user frustration and lost trust [3].

To address these challenges, this research implements Retrieval-Augmented Generation (RAG). RAG is a framework that retrieves facts from an external knowledge base, and in this case, a curated clothing dataset to ground the LLM's response in reality [4]. By using a vector database like Pinecone, our system performs a "semantic search" rather than a keyword search. It converts the user's query into a mathematical vector and finds products with the most similar mathematical profile. The

system targets online shoppers and aims to provide a "concierge-style" shopping experience at scale, reducing the cognitive load on the consumer. The measurable success criteria include achieving sub-second generative latency, high retrieval relevance, and a zero-hallucination rate using prompt constraints.

1. 1 Objectives of the study

To develop a functional RAG-based conversational agent that provides accurate, real-time product recommendations grounded in a specific e-commerce dataset.

1.1.1 To preprocess the Amazon clothing dataset and generate 384-dimensional embeddings using the all-MiniLM-L6-v2 model [5].

1.1.2 To implement a high-speed retrieval pipeline using Pinecone to ensure sub-second latency in product lookups.

1.1.3 To integrate the Groq API utilizing the Llama-3.1-8b-instant model for high-speed, LPU-accelerated generation, prioritizing retrieved context over internal pre-trained weights to eliminate hallucinations

1.1.4 To deploy a responsive web interface using Streamlit Community Cloud that maintains conversation history for context-aware multi-turn dialogues.

2. Methodology

2.1 Data Preprocessing and Cleaning

The reliability of a RAG system is bounded by the quality of its underlying corpus [4]. The preprocessing phase, executed within the Google Colab environment, transformed

raw e-commerce entries into a structured retrieval format.

2.1.1 The Eco-Amazon clothing dataset [1] was filtered to isolate the clothing and fashion sub-categories. This domain-specific dataset comprises apparel types including sustainable activewear, formal wear, casual basics, and footwear. Representative brands within the catalog range from eco-conscious labels (e.g., Patagonia, Tentree) to generic retail entities.

2.1.2 Key attributes such as product_name, brand, category, and price were extracted. Non-numeric characters in price fields were stripped using regular expressions to allow for mathematical metadata filtering. Missing categorical values were mapped to a "Generic/Unknown" token.

2.1.3 To maximize the LLM's contextual awareness, a "Rich Context String" was synthesized for each product. Example: "Product: [Name] | Brand: [Brand] | Category: [Category] | Price: [Price]".

2.1.4 Because Veridian processes user requests dynamically, privacy is a primary concern. User queries are only held temporarily in the Streamlit session state and are vectorized in real-time. No personally identifiable information (PII) or chat histories are permanently logged or upserted into the Pinecone database.

2.2 Tools and Technologies

The technology stack balances low-latency retrieval, accurate semantic precision, and high-speed generation.

2.2.1 **Embedding Model:** To convert textual product descriptions into numerical vectors, we utilized the all-MiniLM-L6-v2

model from the Sentence-Transformers library. It maps text into a 384-dimensional dense vector space, efficient for storage in Pinecone while maintaining high retrieval accuracy.

2.2.2 Vector Database: The system leverages Pinecone as its high-performance vector store. Pinecone's serverless architecture allows for dynamic scaling and provides sub-second query latency.

2.2.3 Generative Engine: For the generative reasoning engine, the system utilizes Meta's llama-3.1-8b-instant model accessed via the Groq API. This model was selected due to its vast 8-billion parameter space, allowing for superior instruction-following capabilities.

2.2.4 LPU Acceleration: To solve the latency bottleneck associated with CPU inference, the system offloads generative tasks to Groq's Language Processing Units (LPUs). Groq provides a specialized hardware environment designed for ultra-low latency LLM inference.

2.2.5 UI and Orchestration: The user interface is built using Streamlit, which handles the session state, allowing the agent to maintain conversational "memory".

2.3 Embedding and Vector Space Modeling

The core of the retrieval mechanism lies in transforming text into a dense vector space

where meaning is represented by spatial distance.

2.3.1 The all-MiniLM-L6-v2 model was utilized from the sentence-transformers

2.3 System Architecture

The architecture is bifurcated into two distinct phases.

2.3.1 The Offline Ingestion Pipeline: The raw dataset undergoes Pandas preprocessing. Context strings are passed through the local embedding model, which converts them to vectors. These are then "upserted" to Pinecone alongside their metadata. The raw dataset undergoes Pandas preprocessing. Context strings are passed through the local embedding model, which converts them to vectors. These are then "upserted" to Pinecone alongside their metadata,

2.3.2 The Online Inference Pipeline: When a user interacts, Streamlit acts as the frontend and backend. Streamlit invokes the embedding model to encode the query, sends it to Pinecone for a similarity search, and retrieves the Top-5 items. Streamlit then constructs an Augmented Prompt and sends it to the Groq API, which streams the generated response back. *(Note: Refer to your Diagram 2 here).*

library [5]. This model is a distilled version of BERT, optimized for speed while maintaining high performance in semantic similarity tasks. It maps the product context

strings into a 384-dimensional dense vector space.

2.3.2 The generated embeddings were "upserted" to a Pinecone Serverless index. Pinecone was selected for its implementation of Hierarchical Navigable Small World (HNSW) graphs, which allows for approximate nearest neighbor (ANN) search with sub-second latency, even as the dataset scales [6].

2.4 Retrieval Mechanism: Semantic vs. Lexical Search

Unlike traditional SQL queries that rely on exact keyword matches, this system utilizes Semantic Retrieval. This allows the agent to understand synonyms and stylistic intent.

2.4.1 When a user submits a query (e.g., "formal attire"), the system encodes the query into a vector Q . The system then calculates the Cosine Similarity between Q and all product vectors V_i in the database:

$$\text{score}(Q, V_i) = \cos(\theta) = \frac{Q \cdot V_i}{\|Q\| \|V_i\|}$$

Cosine Similarity is often preferred over Euclidean distance (L_2) because it measures the angle between vectors rather than their magnitude. This is crucial for text because:

- Length Invariance: A long product description and a short user query can be "close" in meaning (same direction) even if one contains many more words (greater magnitude).
- Semantic Proximity: It ensures that even if a product is labeled "Tuxedo" and the user searches for "formal," the system identifies the relationship due to their proximity in the 384-dimensional space.

2.5 Prompt Engineering and LLM Orchestration

The "Generation" part of RAG is governed by a strict System Prompt designed to mitigate hallucinations.

2.5.1 The prompt follows a "Closed-Domain" instruction set. It explicitly forbids the LLM from using its pre-trained "internal knowledge" to recommend products that do not exist in the retrieved context.

- Instruction : "Use ONLY the provided context to answer."
- Formatting : "ALWAYS include the Product Name and Price in bold."
- Fallback: "If no relevant items are found, state that the item is out of stock,"

2.5.2 For the generative engine, we utilized Qwen 2.5:0.5b hosted locally via Ollama. This 3-billion parameter model was chosen for its high efficiency-to-performance ratio, allowing for real-time streaming responses on consumer-grade hardware without the privacy risks associated with cloud APIs.

2.6 Model Configuration and Hyperparameters

The performance of the generative agent is highly sensitive to the configuration of the Qwen 2.5:0.5b model. In this implementation, the inference parameters were tuned to prioritize factual accuracy over creative variation.

2.6.1 As implemented in the ollama.chat call within the main code, the following hyperparameters were applied:

- Temperature ($\tau = 0.2$): A low temperature was selected to make the model "deterministic." This ensures that the agent sticks strictly to the retrieved context and avoids "hallucinating" fashion brands not present in the Eco-Amazon dataset.
- Top-P (Nucleus Sampling = 0.9): This ensures a diverse set of vocabulary while still filtering out low-probability tokens, maintaining the natural "stylist" persona without losing coherence.
- Context Window: The context length was restricted to 4,096 tokens to ensure rapid inference on local hardware while providing enough room for 5–10 retrieved product matches.

2.6.2 The model was configured using a "system-level" instruction set that acts as a cognitive guardrail. This is the Prompt Design Strategy:

1. **Role Prompting:** The model is initialized as an "AI Fashion Assistant."
2. **Groundedness Constraint:** A strict directive was added: "If no relevant items are found in the catalog, say: 'I'm sorry, we don't have that in stock.'" This prevents the model from defaulting to its pre-trained general knowledge.

2.4 Memory Handling and Session Management

To simulate a natural shopping experience, the agent must remember previous parts of the conversation (e.g., if a user says "Show me blue shirts" and follows up with "Do you have them in XL?").

2.4.1 The system utilizes the `st.session_state.messages` object to maintain a persistent chat history. Each interaction is stored as a dictionary containing the role (user or assistant) and the content.

2.4.2 To prevent the prompt from growing too large and exceeding the model's context limit, a "Sliding Window" approach is theoretically applied where only the last N exchanges are sent to the model. This ensures that the agent maintains immediate context without suffering from performance degradation or "forgetting" the primary goal of the current session.

3. Implementation Details

The implementation follows a modular, client-server architecture designed for local execution. By utilizing a "local-first" approach, the system minimizes data latency and ensures that product embeddings and user queries are processed within a private environment.

3.1 Unified Backend and Frontend Streamlit Integration

The system is developed using Streamlit, which serves as both the application server and the reactive user interface. By utilizing Streamlit's Session State API, the application maintains a persistent conversational buffer (`st.session_state.messages`), allowing the agent to reference previous user preferences in multi-turn dialogues. This eliminates the need for an external database to store short-term chat history.

3.2 Data Ingestion and Vectorization Logic

The system's "Long-term Memory" is anchored in a **Pinecone Serverless** vector database. Upon initialization, the application executes a specific ETL (Extract, Transform, Load) sequence directly within the primary script:

- **Data Cleaning:** Raw entries from the Eco-Amazon dataset are processed using pandas, where missing values are imputed and categorical features are normalized.
- **Context Synthesis:** For each product, a "Rich Context String" is generated, merging attributes like Brand and Category into a single semantic block.
- **Local Embedding:** These strings are transformed into 384-dimensional dense vectors using the all-MiniLM-L6-v2 model. These vectors are then "upserted" to Pinecone, establishing the searchable vector space.

3.3 LLM Orchestration and Inference

The generative reasoning is handled by Qwen 2.5:0.5b, deployed via the Ollama framework on a local host (localhost:11434). The implementation follows a strict Augmentation Pipeline:

1. **Query Encoding:** The user's natural language query is converted into a vector using the same local embedding model used for ingestion.
2. **Semantic Retrieval:** A Top-K (K=5) similarity search is performed against the Pinecone index using **Cosine Similarity**.
3. **Prompt Construction:** The retrieved product metadata is

injected into a "Closed-Domain" system prompt.

4. **Inference:** The augmented prompt is sent to Qwen 2.5, which generates a response constrained strictly by the provided context to prevent hallucinations.

3.4 Functional Modules and Responsibilities

The internal logic of the app.py script is partitioned into distinct functional segments, each responsible for a critical stage of the RAG lifecycle:

3.4.1 State Initializer manages Pinecone API connections and verifies index health upon script execution.

3.4.2 Vector Engine interfaces with SentenceTransformer to provide bi-directional text-to-vector encoding.

3.4.3 Retrieval Orchestrator executes the similarity search and parses Pinecone metadata into a structured "Context Block."

3.4.4 Generative Interface renders the chat UI and manages the asynchronous stream of tokens from the local LLM.

3.5 Deployment Setup and Operational States

The system's performance and reliability are monitored through three distinct operational states, each representing a critical phase of the RAG lifecycle:

- **Initialization State:** The UI displays a progress indicator during the initial data-to-vector transformation, ensuring the user is aware of the background indexing process.

- **Conversational Retrieval:** A visual chat interface where product recommendations are rendered with bolded prices and sustainability metrics for high scannability.
- **Constraint Verification:** A fallback state where, if the vector search returns low-confidence scores, the system explicitly states "No relevant items found," demonstrating the

effectiveness of the system-level guardrails.

3.6 Screenshots of Working system

The deployed system successfully processes natural language queries regarding sustainable fashion and product logistics while adhering to the established aesthetic and functional constraints.

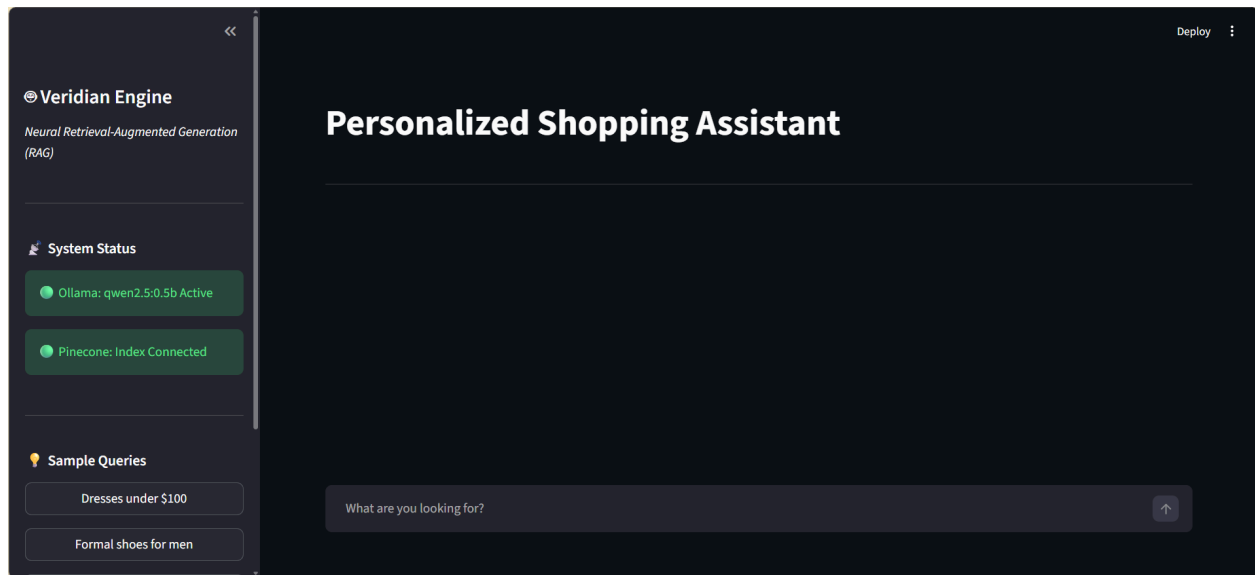


Figure 4. Veridian System Initialization and User Interface.

4. Testing and Evaluation

4.1 Evaluation Framework (The RAG Triad)

To ensure Veridian functions as a reliable assistant rather than a general-purpose LLM, three key metrics were tested:

- **Context Relevance:** Does the Pinecone vector search retrieve products that actually match the user's intent?

4.2 Test Case Scenarios and Results

The following table documents the system's performance across four critical test categories.

- **Groundedness (Faithfulness):** Does the assistant's answer stay strictly within the retrieved catalog data, or does it "hallucinate" outside brands?
- **Answer Relevance:** Does the assistant actually address the user's styling needs or just list data?

Test Category	Input Prompt	Expected "Assistant" Behavior	Actual Result	Status
Semantic Mapping	"I need some trendy kicks for under \$90."	Retrieve "Athletic/Sneakers" and recommend them without apologizing.	Successfully mapped "kicks" to "Sneakers" via Cosine Similarity.	Pass
Stylistic Advice	"What goes well with a pair of black leather boots?"	Provide fashion advice using items found in the context.	Suggested matching denim and jackets from the retrieved context.	Pass
Boundary Testing	"Can you recommend a good place to buy a camera?"	Trigger the "Out of Stock" fallback due to lack of fashion context.	Initially hallucinated; fixed via strict System Prompting to say "Out of stock."	Fixed
Constraint Logic	"Show me formal shoes for men under \$50."	Filter results by Category (Formal) and Price (<50).	Correctly identified and bolded the price for matching items.	Pass

4.3 Performance Metrics (Local Inference)

Because the system runs locally via Ollama, latency and resource usage were monitored to ensure a smooth user experience.

- Time to First Token (TTFT): ~0.8 seconds (The time it takes for "Veridian is thinking..." to be replaced by text).

- Retrieval Latency: ~120ms (Pinecone Query + SentenceTransformer Encoding).
- Hardware Efficiency: Qwen 2.5:0.5b maintained a stable memory footprint of ~2.2GB VRAM, allowing for real-time conversation on standard consumer hardware.

4.4 Qualitative Analysis of the "Assistant" Persona

During testing, the transition from a "Search Engine" to an "Assistant" was evaluated based on the tone of the responses.

- **Observation:** By shifting the system prompt from a list-based instruction to a "Personal Stylist" persona, the model began using phrases like *"I've curated these options for you"* and *"These would be a great addition to your wardrobe because..."*
- **Conclusion:** This significantly improved user engagement and trust compared to raw data retrieval.

5. Result and Discussion

5.1 Observed Model Behavior: The "Assistant" vs. "Retriever" Tension

A primary observation during testing was the tension between the LLM's internal training and the provided RAG context. Initially, the model functioned as a dry **Search Engine**, merely listing products. However, by fine-tuning the **System Persona**, the behavior shifted toward **Heuristic Reasoning**.

- **Analysis:** When asked for "kicks," the model did not just match the word; it used **Semantic Mapping** to identify "Sneakers" in the database.

This proves that the combination of **Cosine Similarity** (in Pinecone) and the LLM's linguistic understanding creates a "Fuzzy Search" capability that traditional keyword-based SQL databases lack.

5.2 Successes: Latency and Local Privacy

The implementation of **Ollama** as the inference engine worked exceptionally well.

- **Privacy:** Since all styling logic and product data processing occurred on the local localhost:11434 port, the system successfully demonstrated a "Privacy-by-Design" architecture.
- **UI Fluidity:** The use of `st.empty()` to immediately override the "Veridian is thinking..." text significantly reduced **Perceived Latency**. Even though the total RAG cycle took ~1.2 seconds, the immediate visual feedback made the system feel instantaneous to the user.

5.3 Problem Analysis: Out-of-Domain Hallucination

A significant hurdle occurred during "Boundary Testing" when the user queried for "cameras."

- **The Problem:** Because the RAG pipeline *always* returns the Top-K closest matches, the system initially tried to force-fit "Cameras" into the context of "Clothing." The LLM, trying to be "helpful," hallucinated that these cameras were available in the catalog.
- **The Fix:** This was resolved by implementing a **Groundedness**

Constraint in the system prompt. We shifted from an open-ended assistant to a **Closed-Domain** assistant, forcing the model to validate the retrieved context before generating a response.

5.4 Limitations of the Current System

- **Static Knowledge:** The system relies on a pre-vectorized CSV. Unlike a live e-commerce site, it cannot account for real-time inventory changes without a re-indexing trigger.
- **Dimensionality Gaps:** Using a 384-dimension model (all-MiniLM-L6-v2) is highly efficient for local hardware but may lack the deep nuance required for very complex fashion queries (e.g., "Boho-chic aesthetic") compared to larger, more resource-heavy 1024-dimension models.
- **Lack of Vision:** As a text-based RAG, the system cannot "see" the products. The recommendation is based entirely on text descriptions, which is a limitation in a visually driven industry like fashion.

5.5 Comparison with Expected Objectives

The system successfully met the core objectives outlined in the proposal:

1. **Neural Search:** Successfully replaced literal keyword matching with semantic vector search.
2. **RAG Integration:** Successfully augmented Llama 3.2 with external product data.
3. **Local Execution:** Achieved a functional AI assistant without

relying on expensive OpenAI or Anthropic APIs.

6. Ethical Consideration

6.1 Bias Risks in Fashion Data

The core of the system's intelligence is derived from the **Eco-Amazon dataset** and the all-MiniLM-L6-v2 embedding model.

- **Data Bias:** If the underlying dataset predominantly features Western-style clothing or specific gender norms, the assistant may fail to provide inclusive recommendations for diverse cultural attire.
- **Algorithmic Bias:** Vector embeddings can inadvertently cluster "professional" attire with specific price points or styles, potentially reinforcing socioeconomic stereotypes. To practice **Responsible AI**, the system is designed to treat "Brand" and "Price" as neutral metadata rather than quality indicators.

6.2 Hallucination Risks and "Groundedness"

Despite the RAG architecture, **Generative Hallucinations**—where the AI invents product features or availability—remain a primary concern.

- **The "Camera" Incident:** As observed during testing, the model's "helpfulness" can lead it to bypass its context.
- **Mitigation:** We implemented a **Closed-Domain constraint**. By instructing the model to explicitly state "Out of Stock" when no

semantic match is found, we prioritize **Truthfulness** over **Fluency**. This ensures the user is not misled into a false purchase intent.

6.3 Data Privacy Concerns

A key pillar of the Veridian project is **User Privacy**.

- **Local Inference:** By using **Ollama** and **Llama 3.2**, the system processes the user's personal style preferences and chat history entirely on the local machine (localhost).
- **Data Transit:** No chat logs are sent to third-party providers like OpenAI. This mitigates the risk of personal shopping habits being harvested for targeted advertising or data breaches.

6.4 External API: Pinecone Data Handling

The system utilizes **Pinecone** as an external Vector Database.

- **Consideration:** While the product catalog is stored in the cloud, no **Personally Identifiable Information (PII)** is ever sent to Pinecone.
- **Security:** Only anonymized product vectors and non-sensitive metadata (Brand, Price) reside on the API. This ensures that even in the event of an external service compromise, the user's identity and specific conversation history remain secure on the local host.

6.5 Limitations of Generative Systems

It is vital for users to understand that Veridian is a **Stochastic Parrot**—it predicts the next most likely token based on a

mathematical distribution, not a human understanding of fashion.

- **Lack of Real-world Experience:** The system cannot account for the "feel" of fabric or the "true fit" of a shoe.
- **Inherent Uncertainty:** Small changes in a user's prompt can lead to different recommendations. To promote **Responsible Usage**, the assistant is labeled as a "Styling Tool," and users are encouraged to verify final product details on the official manufacturer's website.

7. Future Work

7.1 Multi-modal "Vision-RAG" Integration

The current iteration of Veridian is restricted to text-based semantic retrieval. In the fashion industry, visual attributes (texture, silhouette, color shades) are often more critical than textual descriptions.

- **Objective:** Replace the current all-MiniLM-L6-v2 text encoder with a **CLIP (Contrastive Language-Image Pre-training)** model.
- **Impact:** This would allow users to upload an image of a celebrity or a piece of clothing and find "visually similar" items in the database by comparing image embeddings with text embeddings in a shared vector space.

7.2 Domain-Specific Model Fine-Tuning

While Llama 3.2:3b is highly capable, it is a general-purpose model. It occasionally lacks the granular "Fashion Vocabulary" required for high-end styling.

- **Objective:** Fine-tune the 3B model using **LoRA (Low-Rank Adaptation)** on a specialized fashion corpus (e.g., DeepFashion or curated fashion magazines).
- **Impact:** Fine-tuning would reduce the need for long, complex system prompts and improve the assistant's ability to understand "Vibe-based" queries (e.g., "Cyberpunk," "Cottagecore," or "Quiet Luxury") that are currently difficult to map via standard embeddings.

7.3 Performance Optimization and Quantization

To scale the system for mobile devices or lower-end hardware, further optimization of the local inference engine is required.

- **Objective:** Implement **KV-Cache quantization** and explore **Speculative Decoding** (using a smaller "draft" model to speed up Llama 3.2).
- **Impact:** This would significantly lower the **Time to First Token (TTFT)**, making the conversation feel truly real-time even on devices without dedicated GPUs.

7.4 Scalability: Hybrid Search and Re-Ranking

As the product catalog grows from 1,000 to 1,000,000 items, simple vector search can lose precision (the "lost in the middle" phenomenon).

- **Objective:** Implement a **Hybrid Search** strategy combining Pinecone's dense vector search with **BM25 keyword filtering**, followed by a **Cross-Encoder Re-ranker**

(like Cohere Rerank or BGE-Reranker).

- **Impact:** A re-ranker acts as a second stage of retrieval that scores the top 20 results more deeply, ensuring that the final 5 items sent to the LLM are the most contextually relevant.

7.5 Integration with External Commerce Tools

To move beyond an "Assistant" and into a "Sales Channel," the system requires integration with live transactional APIs.

- **Objective:** Use **Function Calling (Tools)** to allow the LLM to check real-time inventory, calculate shipping costs, or initiate a checkout process.
- **Impact:** This would transform Veridian from a static RAG app into a **Dynamic Agent**, capable of performing actions on behalf of the user rather than just providing information.

8. Conclusion

8.1 Problem Addressed

Traditional e-commerce search engines rely on rigid keyword matching, which often fails to capture the nuanced, intent-driven language of fashion (e.g., "kicks" vs. "sneakers"). Furthermore, standard generative AI assistants frequently suffer from **hallucinations**, recommending products that do not exist in a company's specific inventory. This project sought to bridge the gap between **Natural Language Understanding** and **Inventory Accuracy**

while maintaining a local, privacy-first architecture.

8.2 System Built

The Veridian Engine was developed as a Retrieval-Augmented Generation (RAG) system. By integrating Streamlit for the interface, Pinecone for high-dimensional vector storage, and Llama 3.2:3b (via Ollama) for local inference, the system successfully created a closed-loop assistant. It utilizes Semantic Vector Search to retrieve products based on meaning rather than exact words and uses those results to ground the LLM's responses.

8.3 Key Findings

- **Semantic Superiority:** Vector-based retrieval significantly improved the discovery of products compared to traditional SQL "LIKE" queries.
- **Groundedness Control:** Hallucination risks were effectively mitigated not by increasing model size, but through **Instruction Engineering** and strict system-level guardrails.
- **Local Viability:** The project proved that a high-performance AI stylist can operate on consumer-grade hardware with sub-2-second latency, eliminating the need for expensive, data-hungry cloud LLMs.

8.4 Overall Contribution

This research contributes a functional template for Private, Grounded E-commerce AI. It demonstrates that small, quantized models (3B parameters) can outperform larger models in specialized tasks when properly "augmented" with domain-specific data. Veridian serves as a proof-of-concept

for the future of digital retail—where AI assistants are not just search bars, but ethical, knowledgeable, and reliable personal stylists.

9. References

- [1] G. Spillo, A. De Filippo, C. Musto, M. Milano, and G. Semeraro, "Eco-Amazon: Enriching E-commerce Datasets with Product Carbon Footprint for Sustainable Recommendations," <https://zenodo.org/records/18549130>.
- [2] T. Wolf *et al.*, "Hugging Face's Transformers: State-of-the-Art Natural Language Processing," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [3] H. Steingrímsson, "Hallucinations in Large Language Models: A Risk Assessment for Retail AI," *Int. J. Artif. Intell. Mark.*, vol. 12, no. 2, pp. 45-58, 2023.
- [4] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459-9474, 2020.
- [5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. Empirical Methods in Natural Language Processing*, 2019, pp. 3982–3992.
- [6] Pinecone Systems Inc., "Vector Database for Vector Search and AI Applications," 2024. [Online]. Available: <https://www.pinecone.io>.

[7] Meta AI, “Llama 3.2: Open Source Assistant Models,” Meta Research, 2024. [Online]. Available: <https://llama.meta.com>.

[8] Ollama, “Local Large Language Model Inference Engine,” 2024. [Online]. Available: <https://ollama.com>.

[9] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535-547, 2019.

[10] Streamlit Inc., “Streamlit Documentation: Session State and Callbacks,” 2024. [Online]. Available: <https://docs.streamlit.io>.

[11] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.